GMCORE Dashboard 操作手册

GMCORE 开发组

2026-04-20

Contents

GMCORE Dashboard 操作手册 2
目录 2
1. 概述 3
1.1 设计目标 3
1.2 功能模块一览 3
2. 系统需求与环境配置 3
2.1 硬件需求 3
2.2 操作系统 3
2.3 软件依赖栈 4
2.4 一键环境配置 4
3. 安装部署 5
3.1 作为 GMCORE 子模块获取 5
3.2 安装 Python 包（开发模式） 5
3.3 路径配置机制 5
4. 启动与运行 5
4.1 启动命令 5
4.2 端口选择策略 6
4.3 浏览器访问 6
5. 可视化模块 6
5.1 功能概述 6
5.2 操作流程 6
5.3 多变量对比网格 7
5.4 算例分析面板 7
6. 数值试验管理模块 8
6.1 概念模型 8
6.2 界面操作 8
6.3 试验注册表格式 8
7. 多场对比模块 9
7.1 功能定位 9
7.2 操作方法 9
7.3 布局配置 10
8. 参数配置模块 10
8.1 功能概述 10
8.2 操作流程 10

8.3 模板克隆 10
9. 编译与运行模块 10
9.1 编译配置 10
9.2 运行配置 11
9.3 作业管理 11
10. 运行监控模块 11
10.1 实时日志 11
10.2 运行进度 11
10.3 输出文件发现 11
10.4 系统信息 12
11. 试验管理命令行接口 12
11.1 命令总览 12
11.2 create 命令详解 12
11.3 sweep 命令详解 12
11.4 diag 命令详解 13
12. 国际化与界面语言切换 13
12.1 语言支持 13
12.2 切换方式 13
12.3 技术实现 13
13. 架构与扩展 14
13.1 包结构 14
13.2 扩展指南 14
14. 故障排查 15
14.1 常见问题 15
14.2 日志与调试 15
14.3 重置试验数据库 15

GMCORE Dashboard 操作手册

版本： 0.1.0
适用对象： GMCORE 框架使用者、大气动力学数值模式研发人员
最后更新： 2026-04-20

目录

1. 概述
2. 系统需求与环境配置
3. 安装部署
4. 启动与运行
5. 可视化模块
6. 数值试验管理模块
7. 多场对比模块
8. 参数配置模块
9. 编译与运行模块
10. 运行监控模块
11. 试验管理命令行接口

12. 国际化与界面语言切换
13. 架构与扩展
14. 故障排查

---

1. 概述

GMCORE Dashboard 是面向 GMCORE（Grid-point Model dynamical CORE）框架的集成化数值试验管理与诊断
Python Dash 框架为基础构建 Web 交互界面，提供从模式编译、算例配置、参数敏感性扫描、运行提交、实时监控

1.1 设计目标

- 降低操作门槛：将 Fortran namelist 参数编辑、MPI 进程调度、NetCDF 输出诊断等操作封装为图形化界面，
- 支撑参数调优工作流：通过试验注册表、全因子扫描、标量度量提取与双试验对比分析，系统化地组织参数敏
- 可移植性：以独立 Python 包形式发布，通过 Git 子模块机制与 GMCORE 主框架解耦，支持 `pip install -e` 开发模式安装。

1.2 功能模块一览

模块	标签页标识	核心能力
可视化	`visualize`	单变量场渲染（经纬度投影、经向/纬向剖面）、多
数值试验	`experiments`	试验注册表管理、参数扫描配置、运行状态追踪、
多场对比	`multi_view`	跨试验/跨算例的多面板并排对比，支持异源数据
参数配置	`configure`	Namelist 文件在线编辑、模板克隆
编译与运行	`build_run`	CMake 配置、Make 编译、MPI 算例提交
运行监控	`monitor`	进程实时日志、运行进度标记、输出文件发现

---

2. 系统需求与环境配置

2.1 硬件需求

项目	最低要求	推荐配置
CPU	x86_64，2 核	8 核以上（MPI 并行编译与运行）
内存	4 GB	16 GB 以上（视网格分辨率）
磁盘	10 GB（含编译产物）	100 GB 以上（含 NetCDF 输出）
网络	首次安装需要互联网	—

2.2 操作系统

- Linux（推荐 Ubuntu 20.04+、CentOS 7+、WSL2）
- macOS 12+（实验性支持）

2.3 软件依赖栈

2.3.1 Fortran 编译环境

组件	版本要求	用途
GNU Fortran (gfortran)	⩾ 12	Fortran 源码编译
OpenMPI 或 MPICH	⩾ 4.1	消息传递接口运行时及编译器封装（mpifort、mpiexec）
CMake	⩾ 3.16	构建系统生成器
HDF5	MPI-enabled	并行科学数据格式底层库
NetCDF-C	MPI-enabled（`nc-config --has-parallel4 = yes`）	网络通用数据格式 C 库
NetCDF-Fortran	MPI-enabled	NetCDF Fortran 绑定

2.3.2 Python 运行时

组件	版本要求
Python	⩾ 3.10
dash	⩾ 2.0
dash-bootstrap-components	⩾ 1.0
dash-ace	⩾ 0.2
plotly	⩾ 5.0
numpy	⩾ 1.21
xarray	⩾ 2023.1.0
netCDF4	⩾ 1.6.4
f90nml	⩾ 1.4
pyyaml	⩾ 6.0
diskcache	任意
markdown	任意

2.4 一键环境配置

本仓库提供 `environment.yml`（conda 环境定义）和 `setup_env.sh`（自动化安装脚本），可一次性完成全部依赖安装

```
cd /path/to/gmcore/tools/gmcore_dashboard
./setup_env.sh
```

该脚本执行以下操作：

1. 检测包管理器：按优先级探测 mamba → micromamba → conda。
2. 创建或更新环境：根据 `environment.yml` 创建名为 `gmcore` 的 conda 环境，所有 I/O 库强制选择 MPI 并行变体（`hdf5=*=mpi*`、`libnetcdf=*=mpi*`、`netcdf-fortran=*=mpi*`）。
3. 完整性校验：逐项验证 gfortran、mpifort、mpiexec、CMake、nc-config（`--has-parallel4`）、nf-config、h5pcc/h5cc、Python 是否可用。

可选标志：

标志	行为
`--env-only`	仅创建/更新环境，不执行校验
`--verify`	仅执行校验，不修改环境
`--build`	校验后自动执行 CMake 配置与 Make 编译

---

3. 安装部署

3.1 作为 GMCORE 子模块获取

```
git clone https://gitee.com/dongli85/GMCORE gmcore
cd gmcore
./pull_libs.py --libs dashboard
```

或手动:

```
git submodule update --init tools/gmcore_dashboard
```

3.2 安装 Python 包（开发模式）

```
conda activate gmcore
pip install -e tools/gmcore_dashboard
```

安装完成后，系统 PATH 中将注册以下可执行命令:

命令	入口模块	功能
`gmcore-dashboard`	`gmcore_dashboard.cli:main`	启动 Web 仪表盘
`gmcore-experiments`	`gmcore_dashboard.experiments.cli:main`	试验管理 CLI

3.3 路径配置机制

Dashboard 运行时需感知 GMCORE 源码树的根目录位置（以定位 `src/`、`build/`、`run/GMCORE-TESTBED/` 等关键路径）。解析优先级如下:

1. 命令行参数 `--gmcore-root PATH`
2. 环境变量 `GMCORE_ROOT`
3. 自动检测：从当前包文件位置向上逐级搜索，查找包含 `gmcore` 关键字的 `CMakeLists.txt`

若三种方式均无法确定，程序将抛出 `RuntimeError` 并给出明确提示。

---

4. 启动与运行

4.1 启动命令

```
# 最简形式（自动检测根目录，自动选择可用端口）
gmcore-dashboard
```

```
# 显式指定参数
gmcore-dashboard --gmcore-root /path/to/gmcore --host 0.0.0.0 --port 8151

# 开发模式（Dash 热重载）
gmcore-dashboard --debug

# 不安装直接运行
cd /path/to/gmcore
PYTHONPATH=tools python -m gmcore_dashboard.cli
```

4.2 端口选择策略

当未通过 `--port` 指定端口时，程序执行以下探测逻辑：

1. 从端口 8151 起，逐一尝试绑定至 8199。
2. 若区间 [8151, 8199] 内所有端口均被占用，由操作系统内核分配随机可用端口。
3. 最终绑定的端口号将打印至标准输出（`Dash is running on http://host:port/`）。

4.3 浏览器访问

启动成功后，在浏览器中打开输出的 URL。推荐使用 Chromium 内核浏览器（Chrome、Edge）以获得最佳渲染性

---

5. 可视化模块

5.1 功能概述

可视化模块提供对 GMCORE NetCDF 输出文件的交互式场变量渲染能力，支持二维经纬度投影、经向（lon–lev）剖

5.2 操作流程

5.2.1 算例与文件选择

1. 在顶部「算例选择」下拉框中选择目标算例（显示中文名称，如”斜压波（360×180）“）。
2. 系统自动扫描该算例目录下的 `*.h0*.nc`、`*.h1*.nc` 等 NetCDF 历史输出文件，填充「文件选择」下拉框。
3. 选择目标文件后，系统调用 `inspect_nc()` 解析文件元数据（维度、变量列表、时间步数、是否含垂直层）。

5.2.2 变量与视图配置

控件	说明
变量下拉框	列出文件中所有可绘制变量，按维度分类标注（2D/3D/时间序列
时间步滑块	选择渲染的时间索引，标注刻度标签（步数或物理时间）
层次滑块	对三维变量，选择垂直层索引或剖面切割位置
视图模式	水平面（默认）/ 纬向剖面（lat–lev）/ 经向剖面（lon–lev）

5.2.3 色标控制

模式	行为
自动	根据当前帧数据的最小/最大值自动缩放
全局	扫描所有时间步的极值，确保时间演进中色标一致
手动	用户自行指定 `[zmin, zmax]` 范围
对称	使色标关于零点对称（适用于异常场、涡度等有符号量）

系统内置智能色标推断：根据变量名称模式匹配（如含 `tend`、`diff`、`anomaly` 的变量自动启用发散色标 `RdBu_r`）。

5.2.4 地形等值线叠加

对标注为 `topography` 类的算例（如 `mz`、`ksp15`、`dcmip31` 等含理想地形的测试），系统自动在 `gzs` 的等值线，以直观标示地形强迫位置。

5.3 多变量对比网格

位于可视化模块下方，允许用户在同一文件中选择多个变量进行并排对比渲染。

5.3.1 控件说明

控件	功能
变量复选框	勾选 2 个以上变量以启动网格渲染
列数选择	设定每行显示的子图列数（1/2/3）
时间步滑块	独立于顶部主滑块，控制网格内所有子图的时间索引
层次滑块	独立控制垂直层索引
视图模式	独立选择水平面/纬向剖面/经向剖面

上述时间步、层次、视图控件为该区域独立配置，与顶部主视图的控件互不影响，避免用户在页面间反复滚动。

5.3.2 使用场景示例

- 同一时刻对比 `u`（纬向风）与 `v`（经向风）的水平分布
- 检查 `t`（温度）在不同垂直层的结构差异
- 对比 `ps`（地表气压）与 `gzs`（地表位势）的空间相关性

5.4 算例分析面板

右侧提供 Markdown 编辑器，用户可撰写当前算例的分析笔记。

标签	功能
编辑	Markdown 原文编辑（支持 LaTeX 数学公式 `$...$`、`$$...$$`）
预览	实时 Markdown 渲染预览

编辑内容通过浏览器 `localStorage` 持久化存储，按算例目录索引。切换算例时自动加载对应笔记。支持全屏预览模式

6. 数值试验管理模块

6.1 概念模型

本模块实现了结构化的数值试验生命周期管理:

注册表定义 → 试验实例化 → Namelist 生成 → MPI 运行 → 诊断后处理 → 度量提取 → 对比分析

核心数据结构:

概念	存储位置	说明
注册表	`experiments/registry.yaml`	试验定义（基线、参数、继承关系、扫描轴
试验实例	`experiments/experiments/<id>/`	物化后的 namelist、日志、输出、诊断产物
元数据库	`experiments/experiments/experiments.db`	SQLite 数据库，记录状态、参数、度量

6.2 界面操作

6.2.1 详情标签页　选择列表中的试验后显示:

- 试验元数据（名称、基线、状态、创建/更新时间）
- 已解析参数列表（按 namelist 组分类）
- 运行日志尾部（最近 200 行）
- 诊断产物图片浏览

6.2.2 对比分析标签页　选择两个试验进行:

- 参数差异表：逐参数对比，标注变化项
- 度量差异表：标量诊断指标对比
- 诊断图并排：同名诊断图左右对照

6.2.3 参数扫描标签页　对注册表中声明了 `sweep` 配置的试验:

- 展示扫描轴及各轴取值
- 显示全因子组合产生的子试验列表
- 支持一键批量提交

6.3 试验注册表格式

```yaml
version: 1

defaults:
  workspace_root: tools/gmcore_dashboard/experiments/experiments
  executable: build/gmcore_driver.exe
  mpi_launcher: mpirun
  mpi_ranks: 8
  timeout_s: 21600
  hours_per_sol: 24
  diagnostics_set: core
```

```yaml
  my: 1

baselines:
  mars_gomars_base:
    template: tools/gmcore_dashboard/experiments/templates/gomars_base.nml
    params:
      planet: mars
      physics_suite: gomars_v1

experiments:
  cdod_baseline:
    baseline: mars_gomars_base
    description: "参考 GoMars 沙尘提升试验"
    params:
      run_sol: 720
      alpha_d: 5.0e-11
      tau_thresh: 0.04
      alpha_n: 5.0e-5

  ddl_wsl_sweep:
    parent: cdod_baseline
    sweep:
      strategy: full_factorial
      axes:
        alpha_d: [1.0e-11, 5.0e-11, 1.0e-10]
        tau_thresh: [0.02, 0.04, 0.06]
        alpha_n: [1.0e-5, 5.0e-5]
```

路径解析规则：所有相对路径均以 `GMCORE_ROOT` 为基准拼接。绝对路径原样使用。

继承机制：`parent` 字段指定父试验（继承其全部参数并覆盖），`baseline` 指定基线配置模板。

---

7. 多场对比模块

7.1 功能定位

多场对比模块（Multi-view）提供跨试验、跨算例、跨文件的多面板并排可视化能力，适用于：

- 不同分辨率算例的同一变量对比
- 同一算例不同物理参数化方案的结果比较
- 试验组与控制组的空间场差异分析

7.2 操作方法

1. 点击「+ 添加面板」按钮创建新的可视化面板。
2. 每个面板独立配置：
    - 数据来源：测试集算例 或 数值试验（从试验管理模块获取）
    - 文件选择：选择目标 NetCDF 文件

- 变量选择：选择渲染变量
- 视图模式：水平面 / 纬向剖面 / 经向剖面
- 时间步 / 层次：独立时间和垂直层控制

3. 通过「列数」控件调整布局（1–4 列）。
4. 点击「清空全部」移除所有面板。

7.3 布局配置

顶部控制栏提供全局列数设定，面板按设定列数自动排列。各面板等宽渲染，图幅高度固定为 360px。

---

8. 参数配置模块

8.1 功能概述

提供对 GMCORE testbed 算例 namelist 文件的在线浏览与编辑能力，避免手动编辑文本文件可能引入的格式错误。

8.2 操作流程

1. 在「算例与文件选择」面板中选择算例及其 namelist 文件。
2. 右侧编辑器加载文件内容，显示为格式化的 Fortran namelist。
3. 直接编辑参数值。
4. 点击「保存」写回文件，或「还原」丢弃修改。

8.3 模板克隆

通过「克隆模板」功能，可将当前算例配置复制为新算例：

1. 输入新算例名称。
2. 系统在 testbed 目录下创建新算例目录并复制 namelist。
3. 新算例立即出现在算例列表中。

---

9. 编译与运行模块

9.1 编译配置

参数	说明	可选值
构建类型	CMake 编译优化级别	Release（-Ofast）/ Debug（-O0 -g）
精度	浮点精度	R4（单精度）/ R8（双精度，默认）/ R16（四倍精度）
附加 CMake 参数	传递给 cmake 的额外标志	如 `-DUSE_CAM=ON`

点击「编译」按钮后，系统在后台依次执行：

```
conda run -n gmcore bash -c "
  export CONDA_PREFIX=... NETCDF_ROOT=... CC=mpicc FC=mpifort
  cd build && cmake .. -DCMAKE_BUILD_TYPE=Release
  make -j$(nproc)
"
```

编译日志实时追加显示。

9.2 运行配置

参数	说明
算例选择	多选，从 testbed 扫描到的可运行算例
MPI 进程数	并行进程数（默认 2）

点击「运行选中」后，系统对每个选中算例执行：

```
mpiexec -np <N> <build>/gmcore_driver.exe namelist
```

9.3 作业管理

- 所有编译和运行作业由 `JobManager` 统一调度
- 每个作业分配唯一 UUID 前 8 位作为作业标识
- 支持「终止」操作（发送 SIGTERM 至进程组）
- 日志保留最近 5000 行

---

10. 运行监控模块

10.1 实时日志

- 左侧面板选择活跃作业或历史作业
- 中央「日志控制台」以等宽字体实时显示 stdout 输出
- 支持「实时跟踪」模式（自动滚动至末尾）
- 「复制到剪贴板」一键导出日志内容

10.2 运行进度

对于 GMCORE 输出的进度标记（如时间步打印），系统自动解析并展示：

- 已用时间
- 当前积分步骤标识

10.3 输出文件发现

运行结束后，自动扫描算例目录下的 `*.nc` 文件并列出，可直接跳转至可视化模块查看。

10.4 系统信息

显示当前运行环境信息（主机名、CPU 核数、可用内存、conda 环境路径等）。

---

11. 试验管理命令行接口

除 Web 界面外，试验管理功能亦可通过 CLI 操作，适用于批量脚本化工作流。

11.1 命令总览

```
gmcore-experiments <command> [options]
```

命令	功能
`create <name>`	从注册表创建试验实例（生成 namelist、初始化数据库记录）
`run <experiment_id>`	提交运行指定试验
`sweep <name> [--run]`	展开全因子扫描（可选立即运行）
`list [--status X] [--sweep-id Y]`	列出试验（支持状态/扫描组筛选）
`status <experiment_id>`	查看试验完整元数据
`diag <experiment_id> [--name X | --set-name Y]`	执行诊断后处理
`metrics <experiment_id>`	提取标量度量指标
`compare <left_id> <right_id>`	双试验对比分析

11.2 create 命令详解

```
gmcore-experiments create cdod_baseline --registry path/to/registry.yaml
```

执行步骤：

1. 加载注册表，解析继承链，合并基线与覆盖参数。
2. 计算试验 ID（基于父名称 + 参数哈希的确定性命名）。
3. 读取 namelist 模板，应用参数覆盖（通过 `param_catalog` 自动路由至正确的 namelist 组）。
4. 在 workspace 中创建试验目录，写入 namelist 文件。
5. 在 SQLite 数据库中插入元数据记录。

11.3 sweep 命令详解

```
gmcore-experiments sweep ddl_wsl_sweep --run
```

执行步骤：

1. 读取试验定义中的 `sweep.axes` 配置。
2. 按 `full_factorial`（全因子设计）策略生成所有参数组合。
3. 对每个组合调用 `create` 物化试验实例。
4. 若指定 `--run`，依次提交运行。

对于上述示例（3 × 3 × 2 = 18 种组合），将创建 18 个独立试验实例。

11.4 diag 命令详解

```
gmcore-experiments diag <id> --set-name core
```

诊断配置定义在 `diagnostics.yaml` 中：

```yaml
sets:
  core:
    - cdod610
    - season_t
    - season_u
    - polarcap
    - tauxy
    - t15
    - annual_mean

diagnostics:
  cdod610:
    script: tools/gomars/plot_cdod610.py
    input_glob: "{case_name}.h0*.nc"
    args: ["-f", "{hours_per_sol}", "--my", "{my}"]
    outputs: ["cdod610.png"]
```

系统自动：1. 定位输入文件（通过 glob 模式匹配） 2. 调用指定 Python 脚本 3. 收集产物（PNG 图片） 4. 更新元数据库中的诊断记录

---

12. 国际化与界面语言切换

12.1 语言支持

语言	locale 标识	默认
中文（简体）	`zh`	是
English	`en`	否

12.2 切换方式

点击导航栏右上角语言切换按钮（显示当前可切换的目标语言名称）。语言偏好通过浏览器 `localStorage` 持久化，刷新页面后保持选择。

12.3 技术实现

翻译字典集中定义于 `gmcore_dashboard/i18n.py`，通过 `t(key, locale)` 函数调用。所有 UI 文本（导航栏、标签页名称、表单标签、状态提示、错误信息）均通过此机制渲染。

---

13. 架构与扩展

13.1 包结构

```
gmcore_dashboard/
├── __init__.py           # 包版本声明
├── config.py             # 集中路径配置（GMCORE_ROOT 解析）
├── cli.py                # gmcore-dashboard CLI 入口
├── i18n.py               # 国际化翻译字典
├── pyproject.toml        # 构建配置（hatchling）
├── dashboard/            # Web UI 子包
│   ├── app.py             # Dash 应用工厂（create_app）
│   ├── inspector.py       # NetCDF 文件元数据解析
│   ├── plots.py           # Plotly 图表生成
│   ├── colormaps.py       # 色标管理
│   ├── scanner.py         # Testbed 目录扫描
│   ├── case_meta.py       # 算例元数据（中文名称、地图背景）
│   ├── job_manager.py     # 编译/运行进程管理
│   ├── log_parser.py      # 运行日志解析
│   ├── assets/style.css   # 自定义样式
│   └── tabs/              # 各标签页布局与回调
│       ├── visualize.py
│       ├── experiments.py
│       ├── multi_view.py
│       ├── configure.py
│       ├── build_run.py
│       └── monitor.py
├── experiments/          # 试验管理子包
│   ├── cli.py             # gmcore-experiments CLI 入口
│   ├── models.py          # 数据模型（dataclass 定义）
│   ├── registry.py        # 注册表解析与试验解析
│   ├── store.py           # SQLite 持久化层
│   ├── runner.py          # MPI 进程调度与运行
│   ├── sweep.py           # 全因子扫描展开
│   ├── namelist_io.py     # Namelist 读写与参数覆盖
│   ├── param_catalog.py   # 参数自动路由（扫描 Fortran 源码）
│   ├── diagnostics.py     # 诊断后处理调度
│   ├── metrics.py         # 标量度量提取
│   ├── compare.py         # 双试验对比
│   ├── dashboard_backend.py  # Web 界面查询后端
│   ├── registry.yaml      # 试验注册表
│   ├── diagnostics.yaml   # 诊断定义
│   └── templates/         # Namelist 模板
└── docs/                 # 文档
```

13.2 扩展指南

添加新的诊断脚本

1. 将脚本放置于 GMCORE 树的 `tools/` 下。
2. 在 `diagnostics.yaml` 中添加条目，指定脚本路径、输入模式、参数模板、输出文件名。
3. 将其加入相应诊断集（如 `core`）。

添加新算例的中文名称　编辑 `dashboard/case_meta.py` 中的 `CASE_DISPLAY_NAMES` 字典，键为算例目录名，值为中文显

添加新的翻译条目　在 `i18n.py` 的 `_STRINGS` 字典中添加新键值对，随后在 UI 代码中通过 `t("key_name")` 引用。

---

14. 故障排查

14.1 常见问题

症状	原因	解决方法
`RuntimeError: Cannot determine GMCORE_ROOT`	未设置环境变量且自动检测失败	设置 `export GMCORE_ROOT=/path/to/gmcore` 或使用 `--gmcore-root`
`nc-config --has-parallel4 = no`	NetCDF 安装为串行版本	重新安装 MPI-enabled 版本（`conda install libnetcdf=*=mpi*`）
页面加载后显示「测试集目录未找到」	`run/GMCORE-TESTBED/` 不存在	执行 `./run_tests.py` 或手动克隆 testbed 仓库
可视化选择文件后无变量列表	NetCDF 文件损坏或维度命名不规范	使用 `ncdump -h file.nc` 检查文件结构
编译按钮无响应	conda 环境未正确配置	执行 `./setup_env.sh --verify` 检查环境完整性
试验运行超时	`timeout_s` 设置过短	在 `registry.yaml` 中增大 `timeout_s`

14.2 日志与调试

- Dashboard 后端日志输出至 stdout（启动终端可见）。
- 试验运行日志存储于 `experiments/experiments/<id>/logs/run.log`。
- 使用 `--debug` 模式启动可获得 Dash 详细错误回溯。
- SQLite 数据库可直接使用 `sqlite3 experiments/experiments/experiments.db` 查询。

14.3 重置试验数据库

若数据库损坏或需清空:

```
rm -f tools/gmcore_dashboard/experiments/experiments/experiments.db
# 下次运行时自动重建空数据库
```

本手册随 gmcore-dashboard 版本迭代持续更新。如有问题或建议，请通过项目 Issue 提交。